

# FINAL PROJECT REPORT



## DATABASE FOUNDATIONS FOR BUSINESS ANALYTICS PERSONAL EVENT MANAGEMENTSYSTEM

Group 5

### Group Members:

Kesawalekar, Jaan Ashok  
Kode, Sainath  
Mishra, Neeti  
Narayanan, Rohit  
Shetty, Shreya  
Sutaria, Anaysha Dimple

# Team Member's Contribution –

- Database Designing & ERR Diagram – Rohit Narayanan and Jaan Ashok Kesawalekar
- .SQL file creation - Neeti Mishra and Anaysha Dimple Sutaria
- Set 1 table creation and Data Entry – Neeti Mishra, Anaysha Dimple Sutaria, Rohit Narayanan, Jaan Ashok Kesawalekar, Shreya Shetty, Sainath kode
- Set 2 table creation and Data Entry - Neeti Mishra, Anaysha Dimple Sutaria, Rohit Narayanan, Jaan Ashok Kesawalekar, Shreya Shetty, Sainath kode
- Set 3 table creation and Data Entry - Neeti Mishra, Anaysha Dimple Sutaria, Rohit Narayanan, Jaan Ashok Kesawalekar, Shreya Shetty, Sainath kode
- Query creation for Stored Procedure and View – Neeti Mishra, Anaysha Dimple Sutaria, Rohit Narayanan, Jaan Ashok Kesawalekar, Shreya Shetty, Sainath kode
- Project Write Up – Shreya Shetty and Sainath kode
- Report Consolidation - Neeti Mishra, Anaysha Dimple Sutaria, Rohit Narayanan, Jaan Ashok Kesawalekar, Shreya Shetty, Sainath kode

# Project Write up.

## Introduction:

In today's world, the management of events has become more complex. This is because it deals with many details and people that must be managed by considering planning in a wider perspective. Among complex issues in this landscape lies managing individual-event data which requires integrated approach to harmonize different components. The Personal Event Management System is seen as hope amidst all these difficulties since it aims to change the way people think about an event, and they organize and put them together.

In the age of connection pervading all aspects of our lives, this is more than just a database tool; it has changed the way event management is done. It is the embodiment of a paradigm shift in event organization that shows how innovation and necessity come together to give organizers across industries power through their activities starting from small meetings to big galas.

## Objective:

The primary goal of this project is to develop a robust and versatile database management system tailored for efficiently handling person-event data. It aims to streamline event operations, enhance data security and privacy, offer valuable insights, and improve the overall experience for both event organizers and attendees.

Moreover, this database management system will prioritize scalability, ensuring seamless adaptation to varying event sizes and complexities. By employing advanced analytics and machine learning algorithms, it will facilitate predictive modeling, allowing organizers to forecast attendee preferences and optimize event logistics. Additionally, the system will incorporate user-friendly interfaces and customizable features to empower event planners with intuitive tools for managing registrations, schedules, and personalized communications, fostering a more engaging and tailored event experience.

## Business Overview:

The Personal Event Management System caters to a diverse clientele, including individuals planning significant life events, VIPs, celebrities, destination event organizers luxury markets, hospitality sectors, and interest groups. It facilitates efficient event planning, budget management, vendor engagement, guest tracking, and more.

## Features:

1. **Comprehensive Database Management**: Enables efficient storage and retrieval of event-related data.
2. **Event Specific Customization**: Provides tailored features for different event types and sizes.
3. **Budget Management**: Allows detailed budget tracking for various event components.
4. **Vendor and Partner Engagement**: Facilitates seamless collaboration with vendors and partners.
5. **Guest List Management**: Organizes guest information for different events.
6. **Employee Management**: Tracks and manages employee details involved in event planning and execution.
7. **Security Measures**: Adheres to data protection regulations ensuring privacy and security.
8. **Performance Analysis**: Offers insights for future event planning based on historical data.
9. **Mobile App Integration**: Provides a mobile application for on-the-go access to event details, updates, and networking opportunities.
10. **Resource Allocation**: Manages and allocates event resources such as equipment, catering, and transportation efficiently.

These are just some of the key features offered by this Personal Event Management System. This system with its convenient and practical features will help the users create a well-balanced routine for themselves as well as their organizations. Such a system can be a breakthrough in achieving goals at a personal level and at a broader level like various departments and even organizations.

- ✓ Some of the key benefits of this kind of system can be:
- ✓ It helps you plan of time.
- ✓ It helps you organize your events in an efficient way.
- ✓ It helps you track your productivity and thus improve.
- ✓ It helps you set the right expectations and meet the goals.
- ✓ It helps you meet the client's expectations and deliver on time (in case of work schedule).
- ✓ It can be shared with others – teammates or family.
- ✓ It can improve your communication (with team or family) in case of a shared schedule.

## Limitations:

Along with offering so many benefits to the users, there could be some limitations to this system. These limitations may be handled under certain conditions, but in some cases, they are bound to exist. Some anticipated limitations are discussed below:

1. **Complexity in Implementation:** Integrating and customizing the system might require technical expertise.
2. **Initial Data Migration:** Transitioning existing event data into the system could be time-consuming.
3. **User Training:** Adequate training might be necessary for users unfamiliar with such systems.

## Success Criteria:

Our project success broadly depends on the following factors:

1. **Efficient Event Operations:** Measurable improvement in event planning and execution times.
2. **Data Security:** Compliance with data protection regulations and zero breaches.
3. **User Satisfaction:** Positive feedback from event organizers, attendees, and stakeholders.
4. **Improved Insights:** Ability to derive actionable insights for future events from data analysis.

# Relational Data Model

## Database schema

This schema outlines the creation of tables (Events, EventType, Budget, Packages, Engaged) along with their attributes, primary keys, and foreign key constraints that establish relationships between tables in the database. This SQL script can be used to create these tables in a database management system (MySQL.).

# Scenarios covered for the database.

The database schema delineates an expansive architecture poised for managing diverse facets of event planning and coordination. The primary entities, such as events, users, engagements, performers, roles, and guest lists, cater to a comprehensive event management system. The Events table serves as the cornerstone, housing pivotal event particulars: name, location, description, timing, attendee count, and budget association. The Budget table harmonizes fiscal elements, meticulously tracking costs for decor, catering, travel, venue, labor, and overall estimation versus actual expenses.

User-centric functionalities pivot around the User and User\_Details tables, encompassing user credentials, personal profiles, and supplementary details essential for event-related activities. Engagements and packages find their essence in the Engaged and Packages tables, orchestrating the alignment of events with suitable packages and recording engagement specifics like dates, venues, and notes.

The schema delves into performer and venue management, encapsulated within the Performer and Venue tables, storing details about artists, their types, contact information, alongside comprehensive venue attributes. The orchestration of roles and employees is streamlined via the Role, Employee, and Has\_Role tables, delineating roles, employee profiles, and salary associations.

The schema transcends into partnership realms with the Partner and Partner\_Role tables, documenting collaborative partners and their designated roles within the ecosystem. The Is\_Partner table captures the intricate web of partnerships interwoven with roles and events, facilitating a comprehensive view of collaborative efforts.

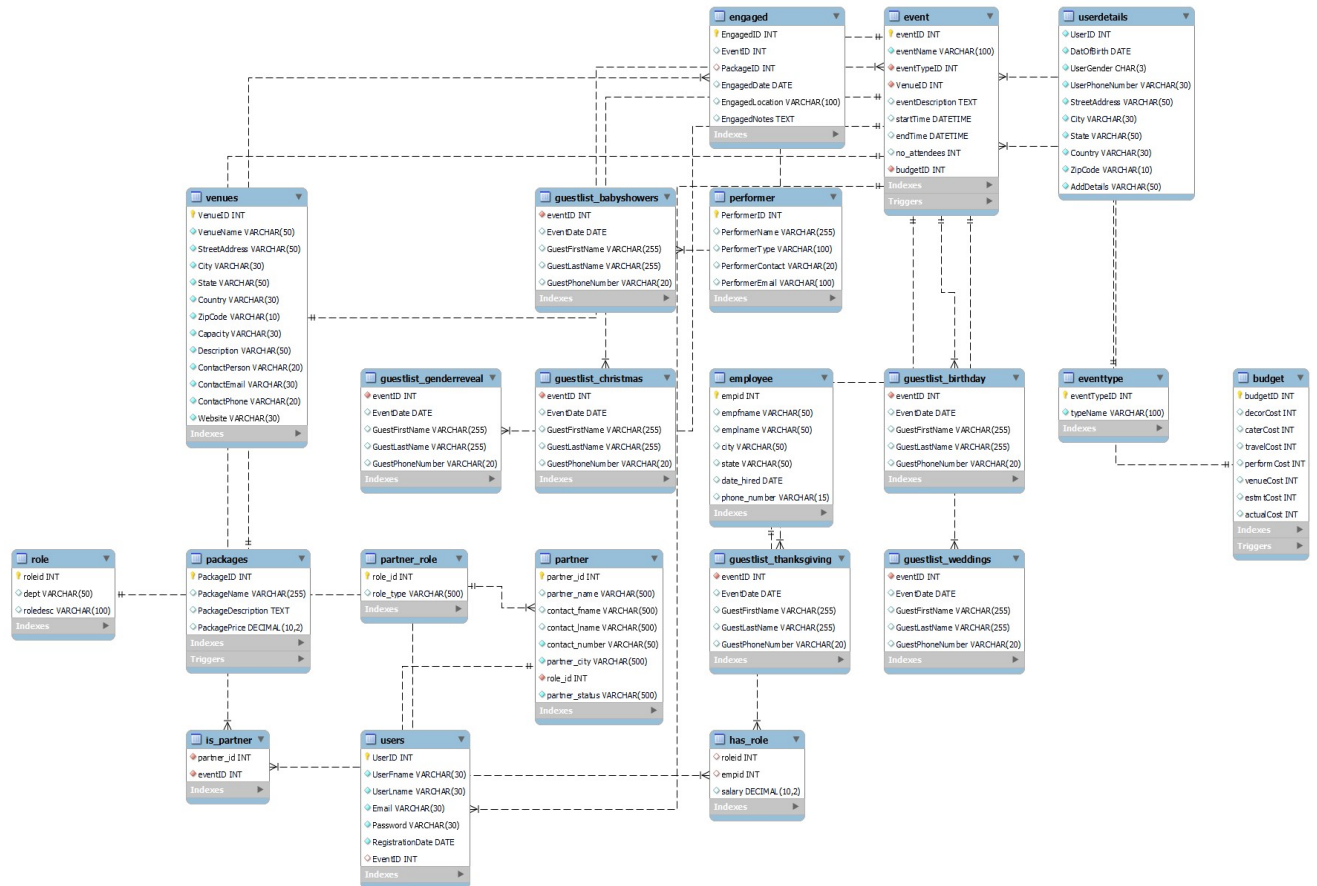
Supplementary tables tailored to specific event types—guest\_list\_birthdays, guest\_list\_Weddings, guest\_list\_Baby\_Showers, guest\_list\_Thanksgiving, guest\_list\_Christmas, and guest\_list\_Gender\_Reveal—ensure meticulous capture of attendee details for varying event categories.

The database offers extensive utility for event planners, enabling streamlined management of event intricacies encompassing logistics, finances, participant details, and partnerships. It lays the groundwork for effective user engagement, financial oversight, and offers a comprehensive canvas for event orchestration and collaboration. Potential enhancements revolve around reinforcing data integrity, optimizing performance, and broadening functionality for user interaction, reporting, or analytical insights.

# Implementation & Evaluation:

|         |                                                              |
|---------|--------------------------------------------------------------|
| Week 1  | Team Meeting and Ideation                                    |
| Week 2  | Topic objective and plan Finalization                        |
| Week 3  | Implementing and Assigning tasks                             |
| Week 4  | Creating Tables                                              |
| Week 5  | Insertion of Data                                            |
| Week 6  | Finalizing Stored Procedures, Triggers, and Functions.       |
| Week 7  | Queries                                                      |
| Week 8  | Implementation of Triggers, Stored Procedures, and Functions |
| Week 9  | Video Preparation & Presentation                             |
| Week 10 | Code Testing                                                 |
| Week 11 | Final Presentation and Report                                |

# ERR Diagram





# Design of the Database

| sr.no | Table Name   | Primary Key                  | Foreign Key(s)                                  | Non-Key Attributes                                                                                                        |
|-------|--------------|------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 1     | Events       | eventID<br>(Auto-increment)  | -                                               | eventName, eventLocation, eventDescription, startTime, endTime, no_attendees                                              |
| 2     | EventType    | eventTypeID                  | -                                               | typeName                                                                                                                  |
| 3     | Budget       | budgetID<br>(Auto-increment) | eventID (to Events)                             | decorCost, caterCost, travelCost, performCost, venueCost, labourCost, estmtCost, actualCost                               |
| 4     | Packages     | packageID                    | -                                               | packageName, packageDescription, packagePrice                                                                             |
| 5     | Engaged      | engagedID                    | eventID (to Events),<br>packageID (to Packages) | engagedDate, engagedLocation, engagedNotes                                                                                |
| 6     | Performer    | performerID                  | -                                               | performerName, performerType, performerContact, performerEmail                                                            |
| 7     | User         | userID                       | eventID (to Events)                             | username, password, email, firstName, lastName, registrationDate                                                          |
| 8     | User_Details | userID                       | -                                               | dateOfBirth, gender, phone, streetAddress, city, state, country, zipcode, additionalInfo                                  |
| 9     | Venue        | venueID                      | eventID (to Events)                             | venueName, streetAddress, city, state, country, zipcode, capacity, description, contactPerson, contactEmail, contactPhone |

|    |                         |           |                                                                     |                                                            |
|----|-------------------------|-----------|---------------------------------------------------------------------|------------------------------------------------------------|
| 10 | Employee                | empID     | -                                                                   | empFname, empLname, city, state, dateHired, phoneNumber    |
| 11 | Role                    | roleID    | -                                                                   | dept, roleDesc                                             |
| 12 | Has_Role                | -         | roleID (to Role),<br>empID (to Employee)                            | salary                                                     |
| 13 | Partner                 | partnerID | -                                                                   | partnerName, contactName, contactNumber, partnerCity       |
| 14 | Partner_Role            | -         | roleID (to Role),<br>partnerID (to Partner)                         | roleName                                                   |
| 15 | Is_Partner              | -         | partnerID (to Partner),<br>roleID (to Role),<br>eventID (to Events) | -                                                          |
| 16 | Guest_List_Birthdays    | -         | eventID (to Events)                                                 | eventDate, guestFirstName, guestLastName, guestPhoneNumber |
| 17 | Guest_List_Weddings     | -         | eventID (to Events)                                                 | eventDate, guestFirstName, guestLastName, guestPhoneNumber |
| 18 | ... (other guest lists) | -         | eventID (to Events)                                                 | eventDate, guestFirstName, guestLastName, guestPhoneNumber |

## 3NF Database

### 1.Events Table:

eventID (Primary Key)

eventName

eventTypeID (Foreign Key referencing EventType)

eventLocation

eventDescription  
startTime  
endTime  
no\_attendees  
budgetID (Foreign Key referencing Budget)

**2.Event Type Table:**

eventTypeID (Primary Key)  
typeName

**3.Budget Table:**

budgetID (Primary Key)  
eventID (Foreign Key referencing Events)  
decorCost  
caterCost  
travelCost  
performCost  
venueCost  
labourCost  
estmtCost  
actualCost

**4.Packages Table:**

packageID (Primary Key)  
packageName  
packageDescription  
packagePrice

## 5.Engaged Table:

engagedID (Primary Key)  
eventID (Foreign Key referencing Events)  
packageID (Foreign Key referencing Packages)  
engagedDate  
engagedLocation  
engagedNotes

# Stored Procedure

## 1.procedure UpdateEmployeeLocation

**Purpose:** The procedure UpdateEmployeeLocation is created to update the city and state information for a specific employee identified by their emp\_id.

### Code :

```
344
345  -- Creatng Procedure For Updating Employee City and State
346  DELIMITER //
347
348  CREATE PROCEDURE UpdateEmployeeLocation(IN emp_id INT, IN new_city VARCHAR(50), IN new_state VARCHAR(50))
349  BEGIN
350      UPDATE Employee
351      SET city = new_city, state = new_state
352      WHERE empid = emp_id;
353  END //
354
355  DELIMITER ;
356
357  CALL UpdateEmployeeLocation(7, 'NewCity', 'NewState');
358
```

Output :

|  | partner_id | partner_name      | contact_fna... | contact_lna... | contact_num... | partner_ci... | role_id | partner_stat... |  |
|--|------------|-------------------|----------------|----------------|----------------|---------------|---------|-----------------|--|
|  | 1          | Indigo Decos      | John           | Doe            | 7363514689     | Dallas        | 1       | Active          |  |
|  | 2          | Opulent           | Grace          | Wilson         | 8265380736     | Dallas        | 2       | Active          |  |
|  | 4          | Eventful Elegance | Alex           | Robbins        | 3746523745     | Michigan      | 4       | Active          |  |
|  | 7          | Morning Frost     | Cassandra      | Reed           | 6341908725     | Seattle       | 4       | Active          |  |
|  | 8          | Dancing Leaves    | Reese          | Miller         | 7162530876     | Minnesota     | 1       | Active          |  |
|  | 9          | Andante           | Weston         | Clark          | 9923471625     | New York      | 5       | Active          |  |
|  | 10         | Victorias         | Jesse          | Adams          | 7652139999     | Dallas        | 3       | Active          |  |
|  |            |                   |                |                |                |               |         |                 |  |
|  |            |                   |                |                |                |               |         |                 |  |

## 2.procedure GetActivePartners()

**Purpose:** The procedure `GetActivePartners` is designed to fetch all details (\*) represents all columns) of partners that have an active status.

code

```
437
438  -- create procedure to fetch only active partners
439  DELIMITER //
440  • create procedure GetActivePartners()
441  ⊖ begin
442    select * from partner where partner_status = 'Active';
443  end //
444  DELIMITER ;
445
446  • call GetActivePartners();
447
```

Output :

|  | partner_id | partner_name      | contact_fna... | contact_lna... | contact_num... | partner_ci... | role_id | partner_stat... |  |
|--|------------|-------------------|----------------|----------------|----------------|---------------|---------|-----------------|--|
|  | 1          | Indigo Decos      | John           | Doe            | 7363514689     | Dallas        | 1       | Active          |  |
|  | 2          | Opulent           | Grace          | Wilson         | 8265380736     | Dallas        | 2       | Active          |  |
|  | 4          | Eventful Elegance | Alex           | Robbins        | 3746523745     | Michigan      | 4       | Active          |  |
|  | 7          | Morning Frost     | Cassandra      | Reed           | 6341908725     | Seattle       | 4       | Active          |  |
|  | 8          | Dancing Leaves    | Reese          | Miller         | 7162530876     | Minnesota     | 1       | Active          |  |
|  | 9          | Andante           | Weston         | Clark          | 9923471625     | New York      | 5       | Active          |  |
|  | 10         | Victorias         | Jesse          | Adams          | 7652139999     | Dallas        | 3       | Active          |  |
|  |            |                   |                |                |                |               |         |                 |  |
|  |            |                   |                |                |                |               |         |                 |  |

## 3.procedure get\_guests :

**Purpose:** The procedure `get_guests` is intended to fetch guests attending a birthday event specified by the eventID.

## code

```
820
821  -- Procedure to fetch guests for a Birthday
822  DROP PROCEDURE get_guests;
823  DELIMITER $$
824  CREATE PROCEDURE get_guests(IN ei INT)
825  BEGIN
826    SELECT * FROM GuestList_Birthday WHERE eventID = ei;
827  END $$
828  DELIMITER ;
829
```

## output

|  | eventID | EventDate  | GuestFirstName | GuestLastName | GuestPhoneNumber |  |
|--|---------|------------|----------------|---------------|------------------|--|
|  | 1       | 2023-10-21 | John           | Doe           | 123-456-7890     |  |
|  | 1       | 2023-10-21 | Jane           | Doe           | 123-456-7890     |  |
|  | 1       | 2023-10-21 | Anne           | Smith         | 123-456-7890     |  |
|  | 1       | 2023-10-21 | Jane           | Smith         | 999-789-9999     |  |
|  | 1       | 2023-10-21 | Alice          | Johnson       | 163-476-7890     |  |
|  | 1       | 2023-10-21 | Janic          | Doe           | 523-456-7890     |  |
|  | 1       | 2023-10-21 | Anne           | Shah          | 123-434-7890     |  |
|  | 1       | 2023-10-21 | Jay            | Day           | 123-456-9090     |  |
|  | 1       | 2023-10-21 | Boot           | Shoe          | 123-456-7434     |  |
|  | 1       | 2023-10-21 | Jae            | De            | 123-456-7890     |  |
|  | 1       | 2023-10-21 | Bnne           | Tmith         | 123-456-7890     |  |
|  | 1       | 2023-10-21 | Zan            | Shiv          | 999-999-9999     |  |
|  | 1       | 2023-10-21 | Ace            | Joon          | 123-456-7890     |  |
|  | 1       | 2023-10-21 | Jac            | De            | 123-478-7890     |  |
|  | 1       | 2023-10-21 | Anne           | Shah          | 123-480-7890     |  |
|  | 1       | 2023-10-21 | Jay            | Day           | 123-956-7890     |  |
|  |         |            |                |               |                  |  |
|  | eventID | EventDate  | GuestFirstName | GuestLastName | GuestPhoneNumber |  |
|  | 7       | 2023-11-01 | Varun          | Doe           | 123-456-7890     |  |
|  | 7       | 2023-11-01 | Jane           | Sharma        | 123-456-7890     |  |
|  | 7       | 2023-11-01 | Harshal        | Smith         | 123-456-7890     |  |
|  | 7       | 2023-11-01 | Jane           | Thakore       | 999-789-9999     |  |
|  | 7       | 2023-11-01 | Anshu          | Johnson       | 163-476-7890     |  |
|  | 7       | 2023-11-01 | Janic          | Shah          | 523-456-7890     |  |
|  | 7       | 2023-11-01 | Anne           | Shah          | 123-434-7890     |  |
|  | 7       | 2023-11-01 | Jay            | Day           | 123-456-9090     |  |
|  | 7       | 2023-11-01 | Boot           | Shoe          | 123-456-7434     |  |
|  | 7       | 2023-11-01 | Jae            | De            | 123-456-7890     |  |
|  | 7       | 2023-11-01 | Bnne           | Tmith         | 123-456-7890     |  |
|  | 7       | 2023-11-01 | Zan            | Shiv          | 999-999-9999     |  |
|  | 7       | 2023-11-01 | Ace            | Joon          | 123-456-7890     |  |
|  | 7       | 2023-11-01 | Jac            | De            | 123-478-7890     |  |
|  | 7       | 2023-11-01 | Anne           | Shah          | 123-480-7890     |  |
|  | 7       | 2023-11-01 | Jayesh         | Day           | 123-956-7890     |  |

# Tigger's

## 1. Tigger calculate\_estmtCost:

**Purpose:** To calculate the estimated cost (estmt Cost) based on various cost components (decor Cost, cater Cost, travel Cost, perform Cost, and venue Cost) and store this calculated value in the estmt Cost field when a new row is inserted into the budget table.

**Event:** The trigger is set to execute before an INSERT operation on the budget table.

**Functionality:** When triggered by an insertion attempt, it performs a calculation by summing up the values of décor Cost, cater Cost, travel Cost, perform Cost, and venue Cost for the new row (NEW row) being inserted. It then assigns this computed total to the estmt Cost field of the same row before it's inserted into the table.

### Code:

```
-- Create Trigger to calculate_estmtCost
DELIMITER //
CREATE TRIGGER calculate_estmtCost
BEFORE INSERT ON budget
FOR EACH ROW
BEGIN
    SET NEW.estmtCost = NEW.decorCost + NEW.caterCost + NEW.travelCost + NEW.performCost + NEW.venueCost;
END;
//
DELIMITER ;
```

### Output :

|  | budgetID | decorCost | caterCost | travelCost | performCo... | venueCost | estmtCost | actualCost |  |
|--|----------|-----------|-----------|------------|--------------|-----------|-----------|------------|--|
|  | 1        | 1000      | 2000      | 500        | 1400         | 1500      | 6400      | 7000       |  |
|  | 2        | 1500      | 2500      | 600        | 1800         | 1600      | 8000      | 8200       |  |
|  | 3        | 1200      | 2200      | 550        | 1200         | 1600      | 6750      | 6930       |  |
|  | 4        | 800       | 1800      | 450        | 1300         | 2000      | 6350      | 6400       |  |
|  | 5        | 2000      | 3000      | 800        | 1300         | 2500      | 9600      | 9800       |  |
|  | 6        | 1300      | 2300      | 600        | 1700         | 1500      | 7400      | 7400       |  |
|  | 7        | 1800      | 2800      | 750        | 2100         | 2000      | 9450      | 10200      |  |
|  | 8        | 900       | 1900      | 500        | 1400         | 2400      | 7100      | 8100       |  |
|  | 9        | 1600      | 2600      | 700        | 1700         | 1900      | 8500      | 8860       |  |
|  | 10       | 1100      | 2100      | 550        | 1500         | 1800      | 7050      | 7425       |  |
|  | 11       | 750       | 1500      | 400        | 1600         | 1200      | 5450      | 5860       |  |
|  | 12       | 900       | 1800      | 500        | 1400         | 1300      | 5900      | 6000       |  |
|  | 13       | 1100      | 2100      | 550        | 1700         | 1500      | 6950      | 6960       |  |
|  | 14       | 1400      | 2400      | 600        | 1300         | 1400      | 7100      | 7800       |  |
|  | 15       | 2000      | 3000      | 800        | 2000         | 2000      | 9800      | 10500      |  |
|  | NULL     | NULL      | NULL      | NULL       | NULL         | NULL      | NULL      | NULL       |  |

## 2. Tigger recalculate \_estmtCost:

**Purpose:** This trigger aims to recalculate and update the estmtCost field in the budget table whenever there's an attempt to update an existing row. It ensures that whenever any of the cost components (decorCost, caterCost, travelCost, performCost, or venueCost) are modified in an existing record, the estmtCost is adjusted accordingly to reflect the new total.

**Event:** The trigger is set to execute before an UPDATE operation on the budget table.

**Functionality:** Like the previous trigger, this trigger recalculates the estmtCost field by summing up the values of decorCost, caterCost, travelCost, performCost, and venueCost for the row being updated (NEW row). It then assigns this new computer total to the estmtCost field of the same row before the update is applied to the table.

**Code :**



```

103
104  -- Creating trigger to calculate the estmt cost again if the values are updated in budget table
105  DROP TRIGGER recalculate_estmtCost;
106
107  DELIMITER //
108  CREATE TRIGGER recalculate_estmtCost
109  BEFORE UPDATE ON budget
110  FOR EACH ROW
111  BEGIN
112      SET NEW.estmtCost = NEW.decorCost + NEW.caterCost + NEW.travelCost + NEW.performCost + NEW.venueCost;
113  END;
114  //
115  DELIMITER ;
116
117
118
119  -- Updating the budget table
120  UPDATE budget
121  SET decorCost = 800, performCost = 1200
122  WHERE budgetID = 1;
123

```

## Output:

|  | budgetID | decorCost | caterCost | travelCost | performCo... | venueCost | estmtCost | actualCost |  |
|--|----------|-----------|-----------|------------|--------------|-----------|-----------|------------|--|
|  | 1        | 800       | 2000      | 500        | 1200         | 1500      | 6000      | 7000       |  |
|  | 2        | 1500      | 2500      | 600        | 1800         | 1600      | 8000      | 8200       |  |
|  | 3        | 1200      | 2200      | 550        | 1200         | 1600      | 6750      | 6930       |  |
|  | 4        | 800       | 1800      | 450        | 1300         | 2000      | 6350      | 6400       |  |
|  | 5        | 2000      | 3000      | 800        | 1300         | 2500      | 9600      | 9800       |  |
|  | 6        | 1300      | 2300      | 600        | 1700         | 1500      | 7400      | 7400       |  |
|  | 7        | 1800      | 2800      | 750        | 2100         | 2000      | 9450      | 10200      |  |
|  | 8        | 900       | 1900      | 500        | 1400         | 2400      | 7100      | 8100       |  |
|  | 9        | 1600      | 2600      | 700        | 1700         | 1900      | 8500      | 8860       |  |
|  | 10       | 1100      | 2100      | 550        | 1500         | 1800      | 7050      | 7425       |  |
|  | 11       | 750       | 1500      | 400        | 1600         | 1200      | 5450      | 5860       |  |
|  | 12       | 900       | 1800      | 500        | 1400         | 1300      | 5900      | 6000       |  |
|  | 13       | 1100      | 2100      | 550        | 1700         | 1500      | 6950      | 6960       |  |
|  | 14       | 1400      | 2400      | 600        | 1300         | 1400      | 7100      | 7800       |  |
|  | 15       | 2000      | 3000      | 800        | 2000         | 2000      | 9800      | 10500      |  |
|  | NULL     | NULL      | NULL      | NULL       | NULL         | NULL      | NULL      | NULL       |  |

## 3.Tigger set\_eventName :

**Purpose:** The trigger set\_eventName is designed to automatically populate the eventName field in the event table when a new row is inserted. It derives the eventName by concatenating the eventID with the typeName from the eventType table based on the eventTypeID provided in the NEW row.

**Functionality:** Upon insertion, the trigger retrieves the typeName associated with the eventTypeID provided in the NEW row and concatenates it with the eventID of the new event. It then assigns this concatenated string to the eventName field of the same row before it is inserted into the event table.

```

167
168 -- Create Trigger to set eventName
169 DELIMITER //
170 CREATE TRIGGER set_eventName
171 BEFORE INSERT ON event
172 FOR EACH ROW
173 BEGIN
174     SET NEW.eventName = (SELECT CONCAT(NEW.eventID, ' - ', typeName) FROM eventType WHERE eventTypeID = NEW.eventTypeID);
175 END;
176 //
177 DELIMITER ;

```

[illegible]

### 1.Function CalculateEventDuration:

- The function `CalculateEventDuration` calculates the duration of an event in hours based on the `startTime` and `endTime` stored in the event table. It takes an `event_id` as input and returns the duration of the event in hours.

### Functionality:

- The function receives an event\_id as a parameter.
- It fetches the startTime and endTime for the specified eventID from the event table.
- Then, it uses the TIMESTAMPDIFF function to calculate the difference in hours between the start Time and end Time and assigns this value to the duration\_hours variable.
- Finally, it returns the calculated duration\_hours.
- d on the star tTime and end Time stored in the event table. It takes an event\_id as input and returns the duration of the event in hours.

## Code

```

202  -- Function to calculate event time duration in hours
203  DELIMITER //
204  • CREATE FUNCTION CalculateEventDuration(event_id INT)
205  RETURNS DECIMAL(8, 2)
206  DETERMINISTIC
207  BEGIN
208      DECLARE start_time DATETIME;
209      DECLARE end_time DATETIME;
210      DECLARE duration_hours DECIMAL(8, 2);
211
212      SELECT startTime, endTime
213      INTO start_time, end_time
214      FROM event
215      WHERE eventID = event_id;
216
217      SET duration_hours = TIMESTAMPDIFF(HOUR, start_time, end_time);
218
219      RETURN duration_hours;
220  END //
221  DELIMITER ;
222  • #Function Call
223  SELECT CalculateEventDuration(4) AS eventDuration;

```

## Output

|  | eventDuration |  |
|--|---------------|--|
|  | 29.00         |  |
|  |               |  |
|  |               |  |
|  |               |  |

## 2.Function CalculateTotalSalary :

**Purpose:** The function CalculateTotalSalary aims to compute the total salary for an employee based on their emp\_id by summing up the salaries associated with roles assigned to that employee.

### Functionality:

- The function takes an emp\_id as a parameter.
- It declares a variable total\_salary to hold the calculated total salary.
- Inside the function, it executes a query using SUM() to add up the salary column from the HasRole table where the empid matches the provided emp\_id.
- The result of the summation is stored in the total\_salary variable.

### Code:

```
482
483  -- Creating Function to Get Total Salary for an Employee
484  DROP FUNCTION CalculateTotalSalary;
485
486  DELIMITER //
487  CREATE FUNCTION CalculateTotalSalary(emp_id INT)
488      RETURNS DECIMAL(10, 2)
489      DETERMINISTIC
490  BEGIN
491      DECLARE total_salary DECIMAL(10, 2);
492
493      SELECT SUM(salary) INTO total_salary
494      FROM Has_role
495      WHERE empid = emp_id;
496
497      RETURN total_salary;
498  END //
499  DELIMITER ;
500
501  SELECT CalculateTotalSalary(8) AS TotalSalary;
502
```

### Output:

|  | TotalSalary |  |
|--|-------------|--|
|  | 180000.00   |  |
|  |             |  |
|  |             |  |
|  |             |  |
|  |             |  |

### 3.function Calculate Discounted Price:

- **Purpose:** The function `CalculateDiscountedPrice` calculates the discounted price for a package based on the `package_id` and a given `discount_percentage`.
- **Functionality:**
  - The function takes two parameters: `package_id` to identify the package and `discount_percentage` to specify the discount percentage.
  - It declares two variables: `original_price` to hold the original price of the package and `discounted_price` to store the calculated discounted price.
  - Inside the function, it retrieves the `PackagePrice` for the specified `package_id` from the `Packages` table and stores it in the `original_price` variable.
  - It calculates the discounted price by subtracting the product of `original_price` and the fraction of `discount_percentage / 100` from the `original_price`.
  - The resulting discounted price is stored in the `discounted_price` variable.
  - Finally, the function returns the `discounted_price`.

code :

```
548
549 -- Function to calculate discounted price
550 DELIMITER //
551 • CREATE FUNCTION CalculateDiscountedPrice(package_id INT, discount_percentage DECIMAL(5, 2))
552 RETURNS DECIMAL(10, 2)
553 READS SQL DATA
554 BEGIN
555     DECLARE original_price DECIMAL(10, 2);
556     DECLARE discounted_price DECIMAL(10, 2);
557
558     SELECT PackagePrice INTO original_price
559     FROM Packages
560     WHERE PackageID = package_id;
561
562     SET discounted_price = original_price - (original_price * (discount_percentage / 100));
563
564     RETURN discounted_price;
565 END //
566 DELIMITER ;
```

output :

|  | DiscountedPrice |  |
|--|-----------------|--|
|  | 135.00          |  |
|  |                 |  |
|  |                 |  |
|  |                 |  |

# Views

## 1.view overbudget\_events :

**Purpose:** The view overbudget\_events is created to provide information about events that have exceeded their estimated budget (estmtCost). It displays details like eventID, eventName, estmtCost, actualCost, and the difference between actual and estimated costs (overbudget\_amount).

**Functionality:**

- It selects columns eventID, eventName, estmtCost, and actualCost from the event and budget tables.

- The JOIN clause links the event table with the budget table based on the budgetID column to associate event details with their respective budget information.
- The WHERE clause filters the results to show only those events where the actualCost exceeds the estmtCost, indicating events that have gone over budget.

### code

```

837  -- View 1: eventID and eventName with Overbudget Information
838  DROP VIEW IF EXISTS overbudget_events;
839  CREATE VIEW overbudget_events AS
840  SELECT
841      e.eventID,
842      e.eventName,
843      b.estmtCost,
844      b.actualCost,
845      (b.actualCost - b.estmtCost) AS overbudget_amount
846  FROM
847      event e
848  JOIN budget b ON e.budgetID = b.budgetID
849  WHERE
850      b.actualCost > b.estmtCost
851  ORDER BY
852      e.eventID;
853
854  SELECT * FROM overbudget_events;

```

### output:

|  | eventID | eventName            | estmtCost | actualCost | overbudget_amo... |  |
|--|---------|----------------------|-----------|------------|-------------------|--|
|  | 1       | 1 - Birthday Party   | 6000      | 7000       | 1000              |  |
|  | 2       | 2 - Christmas Party  | 7100      | 8100       | 1000              |  |
|  | 3       | 3 - Wedding          | 6750      | 6930       | 180               |  |
|  | 4       | 4 - Music Concert    | 7050      | 7425       | 375               |  |
|  | 5       | 5 - Product Launch   | 9600      | 9800       | 200               |  |
|  | 6       | 6 - Fundraiser       | 5900      | 6000       | 100               |  |
|  | 7       | 7 - Birthday Party   | 9450      | 10200      | 750               |  |
|  | 8       | 8 - Gender Reveal    | 7100      | 7800       | 700               |  |
|  | 9       | 9 - Baby Shower      | 8500      | 8860       | 360               |  |
|  | 10      | 10 - Thanksgiving    | 8000      | 8200       | 200               |  |
|  | 11      | 11 - Christmas Party | 5450      | 5860       | 410               |  |
|  | 12      | 12 - Music Concert   | 6350      | 6400       | 50                |  |
|  | 13      | 13 - Wedding         | 6950      | 6960       | 10                |  |
|  | 15      | 15 - Thanksgiving    | 9800      | 10500      | 700               |  |
|  |         |                      |           |            |                   |  |
|  |         |                      |           |            |                   |  |



## 2.view avg\_attendees\_per\_event\_type

**Purpose:** The view avg\_attendees\_per\_event\_type is intended to display the average number of attendees for each event type.

### Functionality:

- It selects columns typeName from the eventType table as eventType and calculates the average number of attendees (AVG(e.no\_attendees)) for each event type.
- The JOIN clause connects the event table with the eventType table using the eventTypeID to associate event details with their respective event types.
- The GROUP BY clause groups the results by typeName (event type) to calculate the average number of attendees for each event type

### code:

```
856
857  -- View 2: Average Attendees per Event Type
858  • DROP VIEW IF EXISTS avg_attendees_per_event_type;
859  • CREATE VIEW avg_attendees_per_event_type AS
860  SELECT
861      et.typeName AS eventType,
862      AVG(e.no_attendees) AS avg_attendees
863  FROM
864      event e
865  JOIN eventType et ON e.eventTypeID = et.eventTypeID
866  GROUP BY
867      et.typeName;
868
869  • SELECT * FROM avg_attendees_per_event_type;
870
```

### output :



|  | eventType       | avg_attendees |  |
|--|-----------------|---------------|--|
|  | Birthday Party  | 60.0000       |  |
|  | Wedding         | 175.0000      |  |
|  | Baby Shower     | 70.0000       |  |
|  | Thanksgiving    | 85.0000       |  |
|  | Gender Reveal   | 100.0000      |  |
|  | Christmas Party | 165.0000      |  |
|  | Product Launch  | 150.0000      |  |
|  | Fundraiser      | 150.0000      |  |
|  | Music Concert   | 2575.0000     |  |
|  |                 |               |  |
|  |                 |               |  |

### 3.view event\_with\_venue :

**Purpose:** The view event\_with\_venue is designed to provide a list of events along with the names of their associated venues.

**Functionality:**

- It selects columns eventID, eventName from the EVENT table, and VenueName from the Venues table.
- The JOIN clause connects the EVENT table with the Venues table based on the VenueID, associating each event with its corresponding venue.

**Code:**

```

871
872  -- View 3: Event with their venues
873 • DROP VIEW IF EXISTS event_with_venue;
874 • CREATE VIEW event_with_venue AS
875 SELECT
876     e.eventID,
877     e.eventName,
878     v.VenueName
879 FROM
880     EVENT e
881 JOIN
882     Venues v ON e.VenueID = v.VenueID
883 GROUP BY
884     e.eventID;
885
886 • SELECT * FROM event_with_venue;
887

```

output :

|  | eventID | eventName            | VenueName                |  |
|--|---------|----------------------|--------------------------|--|
|  | 1       | 1 - Birthday Party   | Crystal Ballroom Suites  |  |
|  | 2       | 2 - Christmas Party  | Golden Horizon Hall      |  |
|  | 3       | 3 - Wedding          | Serenity Plaza           |  |
|  | 4       | 4 - Music Concert    | Harmony Hall             |  |
|  | 5       | 5 - Product Launch   | Regal Renaissance Manor  |  |
|  | 6       | 6 - Fundraiser       | Blissful Harbor Events   |  |
|  | 7       | 7 - Birthday Party   | Grand Garden Pavilion    |  |
|  | 8       | 8 - Gender Reveal    | Starlight Event Center   |  |
|  | 9       | 9 - Baby Shower      | Elegant Oasis Estates    |  |
|  | 10      | 10 - Thanksgiving    | Radiant Skyline Pavilion |  |
|  | 11      | 11 - Christmas Party | Serenity Plaza           |  |
|  | 12      | 12 - Music Concert   | Opulent Oasis Halls      |  |
|  | 13      | 13 - Wedding         | Enchanting Meadows Ve... |  |
|  | 14      | 14 - Product Launch  | Majestic View Venue      |  |
|  | 15      | 15 - Thanksgiving    | Celestial Gardens        |  |
|  |         |                      |                          |  |
|  |         |                      |                          |  |
|  |         |                      |                          |  |

# Conclusion

The Personal Event Management System stands as a comprehensive solution catering to the diverse needs of event organizers across various domains. It aims to simplify event planning, enhance efficiency, and provide valuable insights while prioritizing data security and user satisfaction. With continuous improvement and future expansions, it aims to set new standards in the realm of event management systems.

- By offering a versatile suite of tools and functionalities, this system adapts to the unique requirements of event organizers across diverse industries and event scales. Its flexibility ensures it can cater to the nuances of weddings, corporate conferences, charity galas, and more.
- Through streamlining operations, automating routine tasks, and providing intuitive interfaces, the system drastically reduces the time and effort involved in planning and executing events. This efficiency translates into smoother workflows for organizers.
- The system's ability to convert raw event data into actionable insights empowers organizers to make informed decisions. Historical data analysis allows for better resource allocation, budget forecasting, and strategic planning for future events.
- Prioritizing data security and compliance with stringent regulations assures users of the system's commitment to protecting sensitive information. This instills trust among stakeholders and attendees, fostering stronger relationships.

